

BẢN THẢO KẾ HOẠCH

1. Yêu cầu chung

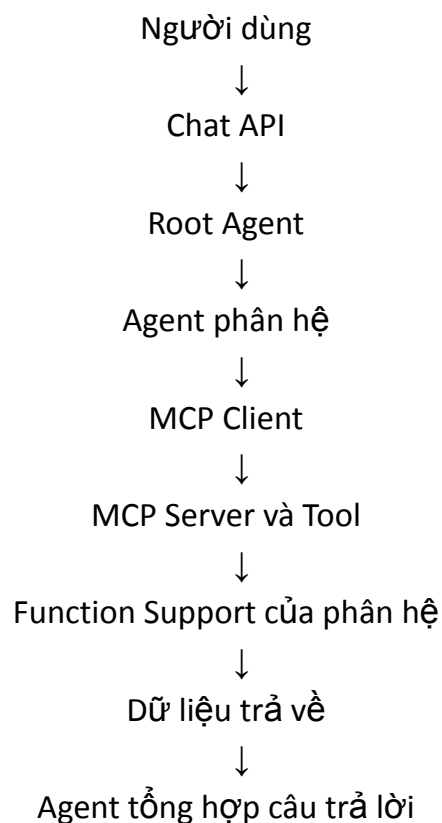
Xây dựng hệ thống AI Agent hỗ trợ truy vấn thông tin tại ba phân hệ:

- Tuyển dụng
- Nhân sự
- Chuyên cần

Số lượng thành viên: 9 người, bao gồm:

- 5 người xây dựng Core Chatbot
- 1 người phụ trách MCP và Agent Tuyển dụng
- 1 người phụ trách MCP và Agent Chuyên cần
- 1 người phụ trách MCP và Agent Nhân sự
- 1 người phụ trách OCR

Mục tiêu của giai đoạn thực tập là hoàn thành một luồng demo:



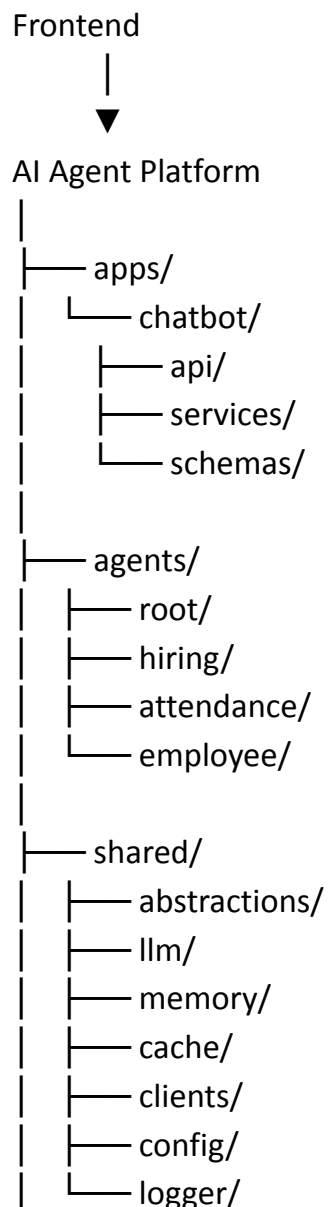
Hệ thống ưu tiên:

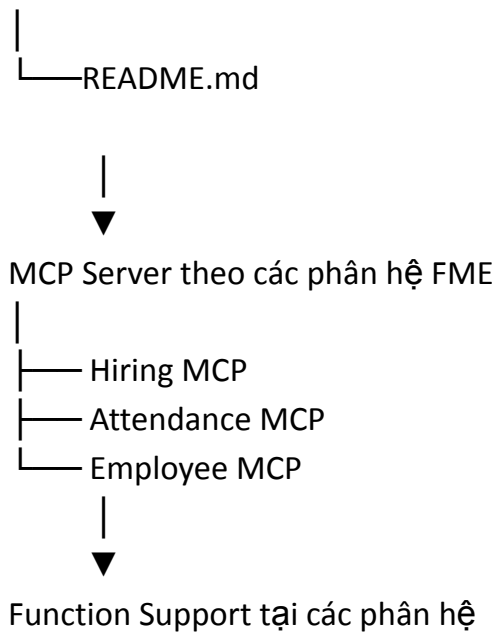
- Truy vấn thông tin có sẵn.
- Trả lời dựa trên dữ liệu từ các phân hệ.
- Demo được luồng hoàn chỉnh từ Chatbot đến MCP.

Chưa ưu tiên:

- Tự động cập nhật hoặc xóa dữ liệu thật.
- Orchestrator nhiều bước phức tạp.
- Deploy production.

2. Kiến trúc tổng quát dự kiến





Nguyên tắc phân chia

- | | | |
|-----------------------|---|------------------------------|
| ❖ Business logic | → | Function Support của phân hệ |
| ❖ MCP giao tiếp | → | MCP Server và MCP Tool |
| ❖ Phân tích yêu cầu | → | Agent của từng phân hệ |
| ❖ Điều phối các Agent | → | Root Agent |
| ❖ Hội thoại và API | → | Core Chatbot |

3. Phân công công việc

3.1. Nhóm Core Chatbot

Nhóm Core Chatbot xây dựng phần nền chung để nhận câu hỏi, quản lý hội thoại, điều phối Agent và kết nối tới các MCP Server.

Thành viên 1 — Root Agent

Công việc

Xây dựng Root Agent làm đầu mối điều phối yêu cầu tới đúng Agent phân hệ.

Ví dụ:

"Có bao nhiêu ứng viên đang chờ phỏng vấn?"



Root Agent



Hiring Agent

Nhiệm vụ cụ thể

- Nhận message và context từ Chat Service.
- Xác định phân hệ phù hợp.
- Chuyển request đến Agent tương ứng.
- Nhận kết quả từ Agent.
- Trả kết quả về Chat Service.
- Quản lý danh sách Agent thông qua Agent Registry.
- Hỗ trợ trường hợp không xác định được Agent.

Ghi chú

- Root Agent không gọi trực tiếp MCP.
- Root Agent không chứa nghiệp vụ Tuyển dụng, Nhân sự hoặc Chuyên cần.
- Chỉ triển khai routing cơ bản trong giai đoạn đầu.
- Cấu trúc và interface của Root Agent cần được mentor hoặc người phụ trách review trước.
- Không tự thêm Planner hoặc workflow phức tạp nếu chưa được thống nhất.

Kết quả cần đạt

- Route được ít nhất ba nhóm intent.
- Có fallback response khi không tìm được Agent.

Thành viên 2 — Memory

Công việc

Xây dựng hệ thống lưu và tải ngữ cảnh hội thoại.

Memory cần hỗ trợ hai mức:

Memory theo người dùng: Dùng chung khi người dùng chuyển qua nhiều Agent

Memory theo Agent: Chỉ lưu ngữ cảnh liên quan đến từng phân hệ

Ví dụ

Người dùng hỏi: "Tìm nhân viên Nguyễn Văn A"

Sau đó hỏi: "Tháng này người đó đi làm bao nhiêu ngày?"

Memory cần giúp Agent hiểu “người đó” là Nguyễn Văn A.

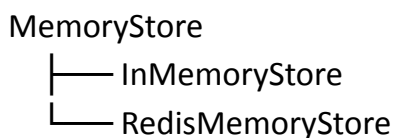
Nhiệm vụ cụ thể

- Lưu conversation và message.
- Tải history theo conversation_id.
- Lưu context chung theo user_id.
- Lưu context riêng theo agent_name.
- Giới hạn số lượng message được đưa vào LLM.
- Có phương án memory in-memory cho giai đoạn đầu.
- Chuẩn bị interface để sau này thay bằng Redis hoặc database.

Ghi chú

Không để Chat Service phụ thuộc trực tiếp vào Redis hoặc database.

Nên thông qua abstraction:



Kết quả cần đạt

- Tạo và tải được conversation.
- Phân biệt được memory giữa các user.
- Phân biệt được context chung và context riêng từng Agent.

Thành viên 3 — LLM Factory

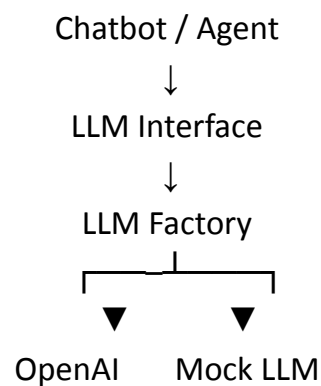
Công việc

Xây dựng lớp dùng chung để khởi tạo và sử dụng các LLM provider.

Nhiệm vụ cụ thể

- Định nghĩa interface chung cho LLM.
- Cho phép chọn provider từ cấu hình.
- Cho phép cấu hình model, timeout và tham số cơ bản.
- Hỗ trợ mock provider để test mà không cần gọi API thật.
- Chuẩn hóa input và output khi gọi LLM.
- Xử lý lỗi cơ bản như timeout hoặc provider không tồn tại.

Kiến trúc dự kiến



Ghi chú

Agent không được tự khởi tạo trực tiếp SDK của provider.

Không nên viết: `client = OpenAI(...)` trong từng Agent.

Thay vào đó: `llm = llm_factory.create(provider_name)`

Kết quả cần đạt

- Chạy được ít nhất một provider thật hoặc provider được team lựa chọn.
- Thay provider bằng config mà không cần sửa Agent.

Thành viên 4 — Chat API và Conversation

Công việc

Xây dựng API đầu vào, đầu ra cho AI Chatbot.

API dự kiến

POST /chat

POST /conversations

GET /conversations/{id}

GET /conversations/{id}/messages

DELETE /conversations/{id}

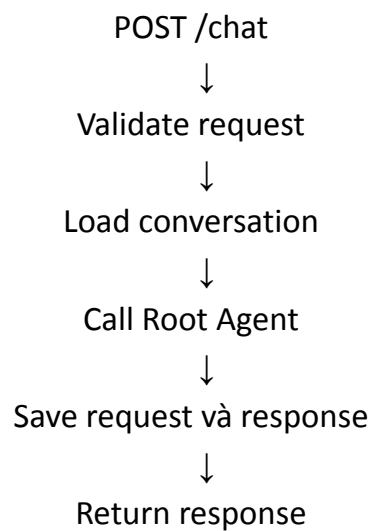
GET /health

...

Nhiệm vụ cụ thể

- Nhận câu hỏi từ người dùng.
- Tạo hoặc sử dụng conversation hiện có.
- Chat Service gọi Root Agent.
- Trả response thống nhất.
- Cung cấp API xem history.
- Tạo tài liệu Swagger/OpenAPI.

Luồng dự kiến



Ghi chú

API không chứa logic chọn Agent.

Controller hoặc router chỉ nên:

- Nhận request.
- Validate.

- Gọi service.
- Trả response.

Kết quả cần đạt

- FE hoặc Postman gọi được /chat.
 - Có Swagger.
 - Có response schema thống nhất.
 - Có test API cơ bản.
-

Thành viên 5 — Cache và MCP Client

Công việc

Xây dựng lớp cache dùng chung và MCP Client để kết nối đến các MCP Server của từng phân hệ.

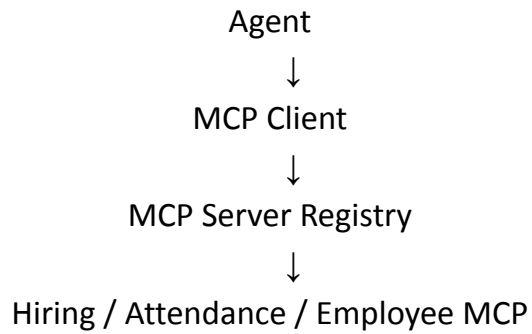
Nhiệm vụ về cache

- Định nghĩa interface cache chung.
- Hỗ trợ cache in-memory trước.
- Cache các dữ liệu phù hợp như:
 - Danh sách Tool.
 - Metadata MCP Server.
 - Kết quả truy vấn ít thay đổi.
- Hỗ trợ thời gian hết hạn.

Nhiệm vụ về MCP Client

- Kết nối đến MCP Server của ba phân hệ.
- Lấy danh sách Tool.
- Gọi Tool theo tên.
- Truyền input đúng schema.
- Chuẩn hóa output trả về cho Agent.
- Xử lý lỗi kết nối và timeout.
- Quản lý thông tin endpoint bằng config.

Luồng dự kiến



Ghi chú

Không hardcode endpoint MCP trong Agent.

Ví dụ cấu hình:

```
mcp_servers:
  hiring:
    endpoint: ...
  attendance:
    endpoint: ...
  employee:
    endpoint: ...
```

Kết quả cần đạt

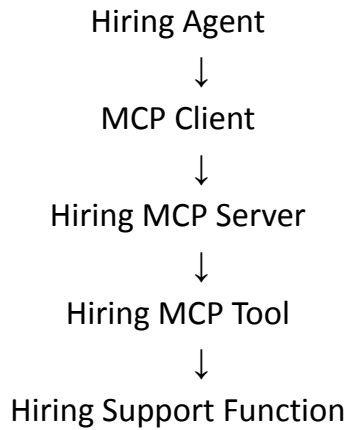
- MCP Client kết nối được ít nhất một MCP Server mock.
- Gọi được Tool theo tên.
- Trả lỗi rõ ràng khi server không hoạt động.
- Sau đó tích hợp lần lượt ba MCP Server thật.

3.2. MCP Server và Agent Tuyển dụng

Mục tiêu

Xây dựng MCP Server và Hiring Agent để hỗ trợ truy vấn thông tin từ phân hệ Tuyển dụng.

Kiến trúc dự kiến



Phần MCP Server

Công việc

- Khởi tạo MCP Server tại vị trí đã được thống nhất.
- Đăng ký các Tool Tuyển dụng.
- Xây dựng input và output schema.
- Validate tham số đầu vào.
- Gọi function support do dev phân hệ cung cấp.
- Chuẩn hóa lỗi trả về.

Tool tham khảo

Chỉ chọn các Tool đã được dev phân hệ xác nhận có function support:

```
search_candidates
get_candidate_detail
list_job_openings
get_interview_schedule
get_recruitment_summary
```

Danh sách này là dự kiến, cần chốt lại theo khả năng thực tế của phân hệ.

Phần Hiring Agent

Công việc

- Nhận yêu cầu từ Root Agent.
- Xác định Tool Tuyển dụng phù hợp.

- Chuẩn hóa input gửi MCP.
- Gọi MCP Client.
- Tổng hợp kết quả thành câu trả lời.
- Xử lý trường hợp thiếu thông tin.
- Trả response thống nhất cho Root Agent.

Ghi chú

- Function support do dev phân hệ phụ trách.
- Giai đoạn đầu ưu tiên Tool chỉ đọc dữ liệu.
- Thống kê chỉ nên dựa trên dữ liệu Tool trả về hoặc function support có sẵn.
- Không tự truy cập database.

Kết quả cần đạt

Demo được một số câu hỏi như:

"Có bao nhiêu ứng viên đang chờ phỏng vấn?"

"Danh sách vị trí đang tuyển là gì?"

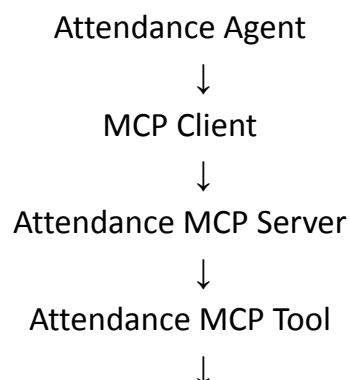
"Cho tôi thông tin ứng viên có mã ..."

3.3. MCP Server và Agent Chuyên cần

Mục tiêu

Xây dựng MCP Server và Attendance Agent để hỗ trợ truy vấn dữ liệu từ phân hệ Chuyên cần.

Kiến trúc dự kiến



Phần MCP Server

Tool tham khảo

get_attendance_history
get_monthly_attendance
get_late_arrival_summary
get_absence_summary
get_attendance_statistics

Các Tool chính thức cần được xác nhận với dev phân hệ.

Công việc

- Khởi tạo MCP Server.
- Đăng ký Tool.
- Định nghĩa input như:
 - Mã nhân viên.
 - Từ ngày.
 - Đến ngày.
 - Tháng và năm.
- Gọi function support.
- Chuẩn hóa output.

Phần Attendance Agent

Công việc

- Hiểu câu hỏi liên quan đến chuyên cần.
- Chọn Tool phù hợp.
- Gọi MCP Tool.
- Tổng hợp dữ liệu thành câu trả lời dễ hiểu.

Ghi chú

- Các phép tính nghiệp vụ nên do function support xử lý, cần trao đổi thêm với dev triển khai phân hệ.

- Agent có thể trình bày hoặc tổng hợp đơn giản từ dữ liệu đã được trả về.

Kết quả cần đạt

Demo được một số câu hỏi như:

"Tháng này nhân viên A đi làm bao nhiêu ngày?"

"Nhân viên A đi trễ bao nhiêu lần?"

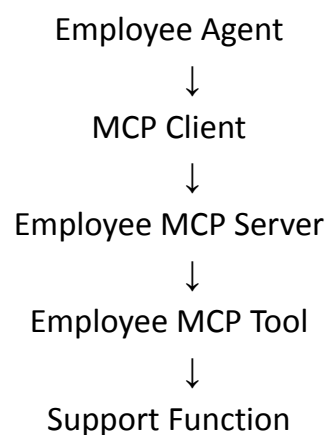
"Cho tôi lịch sử chuyên cần từ ngày ... đến ngày ..."

3.4. MCP Server và Agent Nhân sự

Mục tiêu

Xây dựng MCP Server và Employee/HR Agent để hỗ trợ truy vấn thông tin nhân sự.

Kiến trúc dự kiến



Phần MCP Server

Tool tham khảo

search_employees
get_employee_profile
get_department_list
get_employee_department
get_employee_summary

Cần xác nhận lại các Tool được phép cung cấp và quyền truy cập dữ liệu với dev phân hệ.

Công việc

- Xây dựng MCP Server và Tool.
- Chuẩn hóa input/output.
- Gọi function support.
- Kiểm tra dữ liệu nhạy cảm.

Phần Employee Agent

Công việc

- Nhận câu hỏi từ Root Agent.
- Xác định Tool Nhân sự phù hợp.
- Gọi MCP.
- Tổng hợp và trả lời.
- Không trả dữ liệu ngoài phạm vi được phép.

Ghi chú

Phân hệ Nhân sự có thể chứa dữ liệu nhạy cảm. Cần làm rõ:

- Trường dữ liệu nào được phép truy vấn.
- Role nào được phép xem.
- Dữ liệu nào phải ẩn.
- Có cần nhận thông tin user và permission từ Gateway hay không.

Trong giai đoạn demo có thể sử dụng dữ liệu mock hoặc giới hạn trường dữ liệu.

Kết quả cần đạt

Demo được một số câu hỏi như:

"Tìm thông tin nhân viên có mã ..."

"Nhân viên A thuộc phòng ban nào?"

"Phòng ban X có bao nhiêu nhân viên?"

3.5. OCR

Công việc

Tiếp tục triển khai theo kế hoạch OCR hiện tại.

Các bạn khác khi cần hỗ trợ hoặc cần tích hợp OCR sẽ trao đổi trực tiếp với người phụ trách.

Phạm vi tích hợp dự kiến

OCR có thể được sử dụng như một chức năng độc lập hoặc làm input cho Agent sau này.



Ghi chú

Trong 7 tuần, không nên để tiến độ Core Chatbot hoặc MCP phụ thuộc hoàn toàn vào OCR.

Nếu OCR chưa hoàn thành, Chatbot và Agent vẫn cần demo được bằng input văn bản.

4. Quy ước chung

4.1. Abstract và interface dùng chung

Những thành phần có hành vi giống nhau cần thống nhất interface.

Cấu trúc dự kiến:

shared/
└─ abstractions/

```
|— agent.py
|— memory.py
|— llm.py
|— cache.py
|— mcp_client.py
```

Quy tắc thay đổi

- Không tự sửa abstract/interface dùng chung mà chưa trao đổi.
- Khi cần thay đổi phải thông báo các thành viên liên quan.
- Cần nêu rõ:
 - Lý do thay đổi.
 - Input cũ và mới.
 - Output cũ và mới.
 - Các module bị ảnh hưởng.
- Chỉ thực thi sau khi các bên liên quan đã thống nhất.

4.2. Thống nhất input/output

MCP Tool và Agent cần có schema rõ ràng.

Ví dụ request gọi Tool:

```
{
  "employee_id": "E001",
  "from_date": "2026-08-01",
  "to_date": "2026-08-31"
}
```

Ví dụ output:

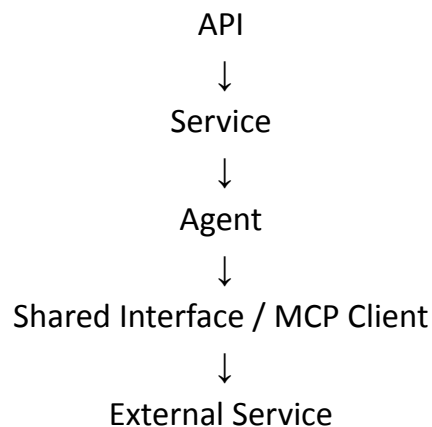
```
{
  "success": true,
  "data": {},
  "error": null,
  "metadata": {
    "source": "attendance"
  }
}
```


4.3. SOLID và OOP

Nhóm Core Chatbot cần lưu ý:

- Một class chỉ nên có một nhiệm vụ.
- Không gọi trực tiếp provider LLM trong Agent.
- Không gọi trực tiếp Redis trong Chat Service.
- Không gọi trực tiếp MCP Server trong API Controller.
- Ưu tiên dependency injection.
- Tách interface khỏi implementation.
- Không đưa business logic vào **shared/**.
- Không copy code giữa các Agent nếu có thể tái sử dụng.

Luồng dependency dự kiến:



5. Tiêu chí hoàn thành

Core Chatbot

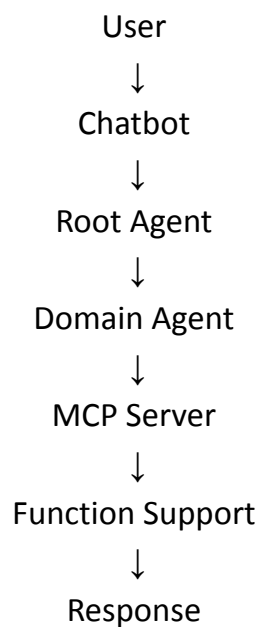
- Có API / chat.
- Có conversation và history cơ bản.
- Có Root Agent route được ba Agent.
- Có Memory interface.
- Có LLM Factory.
- Có MCP Client.
- Có logging và config.
- Có unit test cơ bản.

Mỗi phân hệ

- Có MCP Server chạy được.
- Có ít nhất 2–3 Tool đã thống nhất.
- Có Agent tương ứng.
- Agent gọi được MCP Tool.
- MCP Tool gọi được mock hoặc function support thật.
- Có test và tài liệu Tool.

Toàn hệ thống

Demo được luồng:



6. Ghi chú

Mỗi phân hệ nên ưu tiên 2–3 use case rõ ràng thay vì cố xây nhiều Tool.

Ví dụ:

Tuyển dụng

- Tra cứu ứng viên.
- Tra cứu vị trí tuyển dụng.
- Tổng hợp trạng thái tuyển dụng đơn giản.

Chuyên cần

- Xem lịch sử chuyên cần.
- Tổng hợp đi trễ hoặc vắng mặt.
- Thống kê theo tháng.

Nhân sự

- Tra cứu hồ sơ nhân viên.
- Tra cứu phòng ban.
- Thống kê số lượng nhân viên cơ bản.

Trong kế hoạch này, Core Chatbot và MCP Server được phát triển song song. Agent của từng phân hệ bắt đầu hoàn thiện sau khi MCP Tool đã có tên, input và output tương đối ổn định. Business logic vẫn thuộc function support của các phân hệ FME; MCP là lớp cung cấp Tool, còn Agent chịu trách nhiệm lựa chọn Tool và tổng hợp kết quả cho người dùng.